



UNIVERSIDAD
DON BOSCO

Introducción a JAVA

Programación Orientada a Objetos



Mg. Karens Medrano



Historia de Java

Los Orígenes (1991-1995)

Java nació en 1991 como "Oak" en Sun Microsystems, creado por James Gosling y su equipo. Originalmente diseñado para dispositivos electrónicos de consumo, el proyecto cambió de rumbo cuando el equipo reconoció el potencial de Internet.

En 1995, Java fue oficialmente presentado al mundo.

El slogan "Write Once, Run Anywhere" (Escribe una vez, ejecuta en cualquier lugar) revolucionó el desarrollo de software, prometiendo verdadera portabilidad entre plataformas.

Hoy, Java continúa siendo uno de los lenguajes más utilizados globalmente, respaldando desde aplicaciones móviles Android hasta sistemas empresariales masivos y servicios en la nube.

Evolución y Consolidación

La adopción empresarial fue explosiva. Java 2 (1998) introdujo la plataforma J2EE para aplicaciones empresariales, consolidando a Java como líder en el desarrollo corporativo.

Oracle adquirió Sun Microsystems en 2010, marcando una nueva era. Desde entonces, Java ha evolucionado significativamente con ciclos de lanzamiento predecibles cada seis meses, introduciendo características modernas como módulos, expresiones lambda, records y pattern matching.



Instalación y Configuración de la JDK (Java Development Kit)

El **JDK** (Java **D**evelopment **K**it) son el conjunto de herramientas de Java para desarrollar aplicaciones. Su Link de descarga es: <https://www.oracle.com/java/technologies/downloads/>

Tools and resources

Java downloads

Java archive

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#) (NFTC).

JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License](#) (OTN) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

Linux

macOS

Windows

Product/file description	File size	Download
ARM64 Compressed Archive	203.36 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	202.96 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)

Instalación y Configuración de la JDK (Java Development Kit)



01 Descarga del JDK

Visita el sitio oficial de Oracle o adopta distribuciones alternativas como OpenJDK, Amazon Corretto o Adoptium.

Selecciona la versión LTS (Long Term Support) más reciente para estabilidad a largo plazo.

03 Configuración de Variables de Entorno

Configura JAVA_HOME apuntando al directorio de instalación del JDK. Añade %JAVA_HOME%\bin (Windows) o \$JAVA_HOME/bin (Linux/macOS) a la variable PATH del sistema para ejecutar comandos Java desde cualquier ubicación.

02 Instalación del Paquete

Ejecuta el instalador descargado y sigue las instrucciones en pantalla.

En Windows, el instalador típicamente coloca el JDK en C:\Program Files\Java.

En Linux y macOS, extrae el archivo y muévelo a la ubicación apropiada.

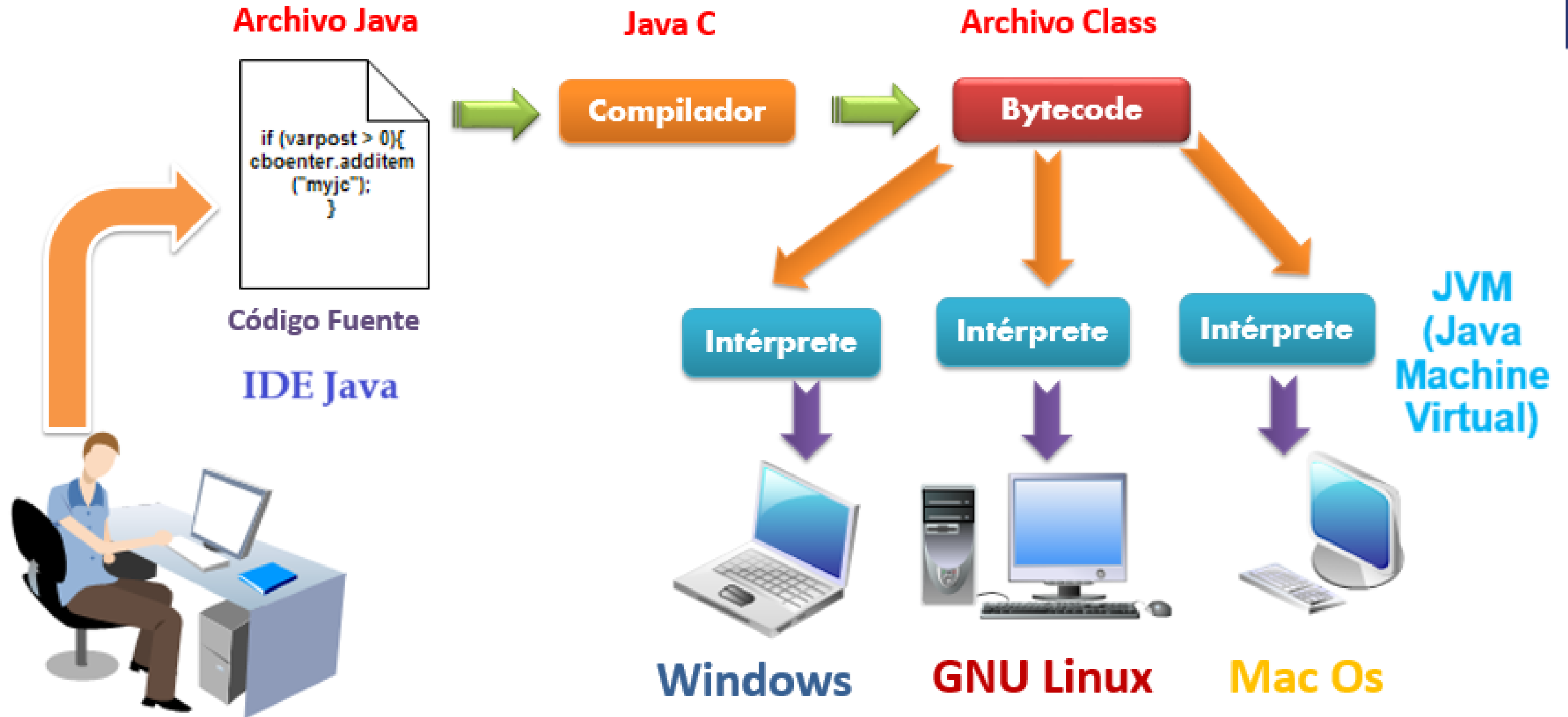
04 Verificación de la Instalación

Abre una terminal o símbolo del sistema y ejecuta "java -version" y "javac -version".

Si ambos comandos muestran la versión instalada, la configuración fue exitosa.

Consejo Pro: Para proyectos profesionales, considera utilizar gestores de versiones como SDKMAN! (Linux/macOS) o Jabba que permiten instalar y cambiar entre múltiples versiones de JDK fácilmente.

Funcionamiento del JDK



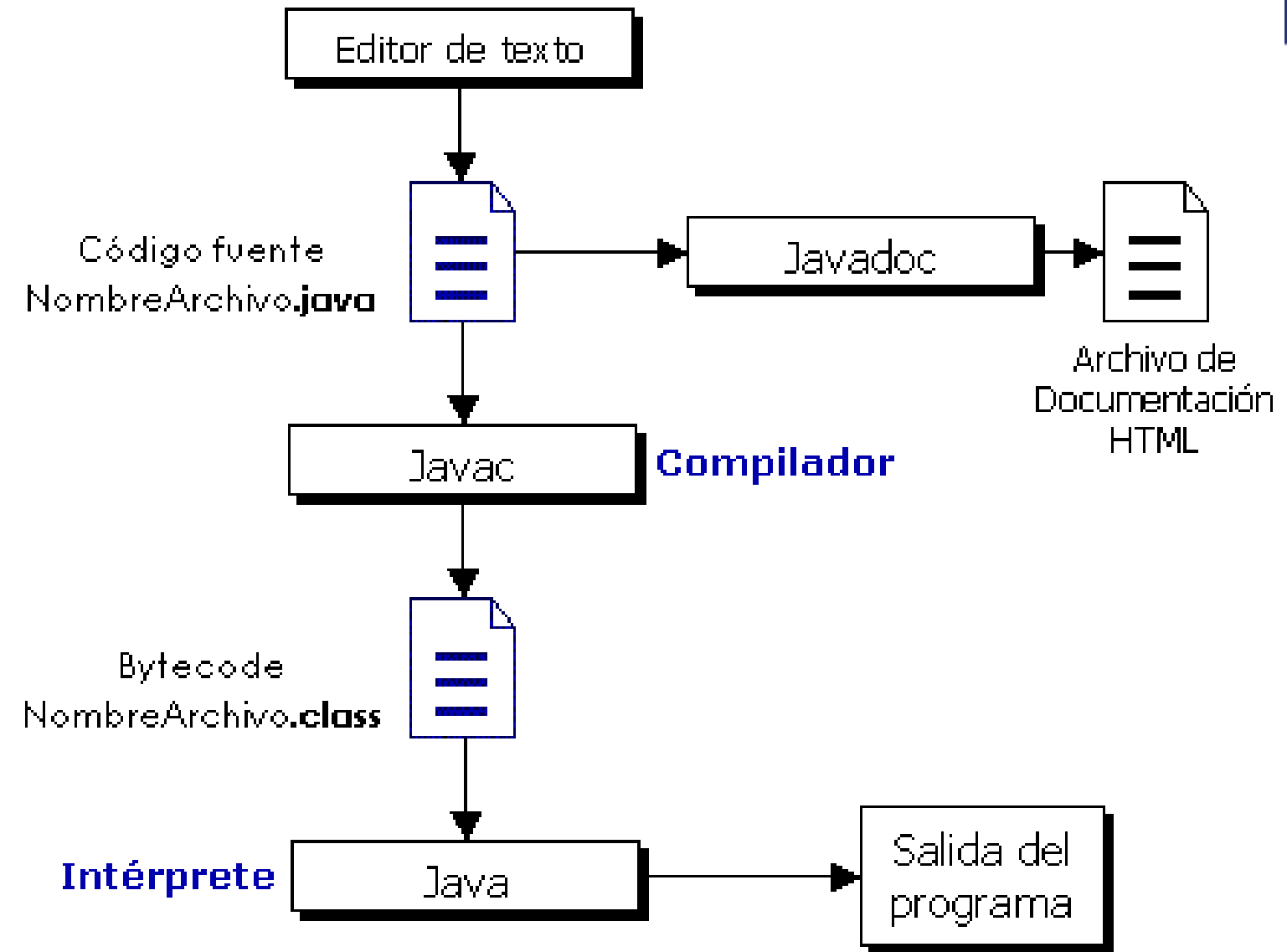


UNIVERSIDAD
DON BOSCO



Compilacion y Ejecucion de Aplicaciones

- **Compilación en el JDK:** proceso que usa el compilador **javac** para convertir un código fuente Java (.java) en un lenguaje intermedio llamado bytecode (.class).
- **javac:** es el compilador de Java, una herramienta de línea de comandos incluida en el JDK que convierte el código fuente (archivos .java) a bytecode (archivos .class), que luego puede ser ejecutado por la Máquina Virtual de Java (JVM).
- **Bytecode:** Es un código intermedio, binario y de bajo nivel, generado por un compilador a partir del código fuente (.java) de un programa.
- El Bytecode no es directamente ejecutable por el hardware, sino por una JVM, permitiendo la portabilidad entre diferentes plataformas sin recompilar, haciendo el código independiente del sistema operativo y la arquitectura de hardware



Características Principales de Java



Independencia de Plataforma

El bytecode de Java se ejecuta en la JVM (Java Virtual Machine), permitiendo que el mismo código funcione en Windows, Linux, macOS y más sin modificaciones.



Orientado a Objetos

Java implementa completamente el paradigma orientado a objetos con clases, herencia, polimorfismo y encapsulación como conceptos fundamentales.



Seguridad Robusta

Incluye múltiples capas de seguridad: verificación de bytecode, gestión de memoria automática y un modelo de permisos granular para aplicaciones.



Gestión Automática de Memoria

El recolector de basura (Garbage Collector) gestiona automáticamente la asignación y liberación de memoria, reduciendo errores de programación y fugas de memoria.



Multithreading Integrado

Soporte nativo para programación concurrente con threads, permitiendo desarrollar aplicaciones eficientes que aprovechan procesadores multi-núcleo.



Biblioteca Estándar Extensa

La API de Java incluye miles de clases predefinidas para tareas comunes: manipulación de archivos, redes, bases de datos, interfaces gráficas y más.

Ventajas y Desventajas de Java



✓ Ventajas

- **Portabilidad excepcional:** El código funciona en cualquier plataforma con JVM instalada
- **Comunidad masiva:** Millones de desarrolladores y abundante documentación
- **Ecosistema maduro:** Frameworks robustos como Spring, Hibernate, y herramientas empresariales
- **Rendimiento optimizado:** La JVM moderna incluye compilación JIT y optimizaciones avanzadas
- **Mantenimiento a largo plazo:** Actualizaciones regulares y soporte LTS garantizado
- **Oportunidades laborales:** Alta demanda en el mercado, especialmente en desarrollo Empresarial
- **Retrocompatibilidad:** El código antiguo generalmente funciona en versiones nuevas

⚠ Desventajas

- **Verbosidad del código:** Requiere más líneas comparado con lenguajes modernos como Python o Kotlin
- **Consumo de memoria:** La JVM y aplicaciones Java tienden a usar más RAM
- **Tiempo de arranque:** Las aplicaciones pueden tardar más en iniciar debido a la carga de la JVM
- **Curva de aprendizaje inicial:** Los conceptos OOP pueden resultar complejos para principiantes
- **UI limitada:** Las interfaces gráficas nativas (Swing/JavaFX) son menos populares que alternativas web
- **Complejidad en configuración:** Proyectos grandes requieren gestión cuidadosa de dependencias

A pesar de estas consideraciones, Java sigue siendo una elección sólida para aplicaciones empresariales, sistemas backend robustos y desarrollo Android. Su madurez y estabilidad compensan ampliamente sus limitaciones.

IDE's Disponibles para Java

IntelliJ IDEA

Considerado el IDE más potente para Java. Ofrece análisis de código inteligente, refactorización avanzada y excelente integración con frameworks. Versión Community gratuita y Ultimate de pago.

Eclipse

IDE de código abierto con una historia larga en el ecosistema Java. Extensible mediante plugins, ideal para desarrollo empresarial. Completamente gratuito y respaldado por la Eclipse Foundation.

NetBeans

IDE oficial de Oracle, ahora proyecto Apache. Excelente para principiantes con asistentes visuales y soporte integrado para Maven. Incluye diseñador GUI visual para aplicaciones Swing.

Visual Studio Code

Editor ligero con extensiones Java potentes. Perfecto para proyectos pequeños o desarrollo rápido. Gratuito, rápido y altamente personalizable con el Extension Pack for Java.

BlueJ

IDE educativo diseñado específicamente para enseñar programación orientada a objetos. Visualización interactiva de clases y objetos. Ideal para estudiantes universitarios iniciándose en Java.



Fundamentos de programación en Java

Sintaxis de Java

.La sintaxis de un lenguaje define cómo se usarán las palabras claves, los operadores y las variables para construir y evaluar expresiones.

La sintaxis de Java especifica como se escribirán los siguientes elementos:

- A. Comentarios
- B. Tipos de datos, Variables, Identificadores
- C. Palabras reservadas
- D. Expresiones y operadores
- E. Bloques y sentencias
- F. Estructuras de Control

Introducción Sintáctica y Comentarios



Estructura Básica de un Programa Java

Todo programa Java comienza con una clase que contiene el método main, punto de entrada de la aplicación:

```
public class MiPrimeraClase {  
    public static void main(String[] args) {  
        System.out.println("¡Hola Mundo!");  
    }  
}
```

Elementos clave:

- `public class`: Declaración de una clase pública
- `main`: Método especial ejecutado al iniciar
- `String[] args`: Argumentos de línea de comandos
- Las llaves `{ }` delimitan bloques de Código
- Las instrucciones terminan con punto y coma `;`

Tipos de Comentarios

Los comentarios documentan el código y son ignorados por el compilador:

```
// Comentario de línea única  
// Se usa para explicaciones breves  
/* Comentario de múltiples líneas  
Es util para bloques de texto o desactivar código  
temporalmente */  
  
/**  
• Comentario de documentación (Javadoc)  
• * @author Tu Nombre * @version 1.0 * @param edad  
  La edad del usuario * @return true si es mayor  
  de edad */
```

Los comentarios Javadoc generan documentación HTML automática del código, esencial en proyectos profesionales.

Convención de Nombres: Las clases comienzan con mayúscula (PascalCase), los métodos y variables con minúscula (camelCase), y las constantes en MAYÚSCULAS_CON_GUIONES.



Fundamentos de programación en Java

Convenciones de nomenclatura en Java(1)

Las comunidades de desarrollo de software en cada lenguaje de programación suelen tener sus preferencias y guías de estilo que dictan cuándo y dónde usar letras mayúsculas y minúsculas. Seguir estas convenciones hace que el código sea más legible y fácil de mantener..

En Java, se utilizan las siguientes convenciones de uso de mayúsculas y minúsculas en los identificadores para distinguir entre variables, métodos, constantes y clases:

- **PascalCase** (CamelloEnMayúsculas):
- **camelCase** (camelloEnMinúscula):
- **UPPER_SNAKE_CASE** (SERPIENTE_EN_MAYUSCULA)



Fundamentos de programación en Java

Convenciones de nomenclatura en Java(2)

PascalCase (CamelloEnMayúsculas):

- La inicial de cada palabra del identificador se escribe en mayúsculas. Se usa para nombrar **clases**, **interfaces**, **registros** y *otros tipos definidos* por el usuario.
- Ejemplos: CostoVentas, AreaTotal, CustomerService, UserProfile, AudioSystem.

camelCase (camelloEnMinúscula):

- Es similar a PascalCase, pero la primera letra de la primera palabra está en minúscula. Se usa para nombrar **variables**, **parametros** y **funciones**.
- Ejemplos: nombreCliente, sumaEdades, customerService, userProfile, audioSystem.

UPPER_SNAKE_CASE (SERPIENTE_EN_MAYUSCULA):

- Se utiliza para identificar a **constantes**. Todas las letras son mayúsculas y las palabras están separadas por guiones bajos. Ejemplos: PI, CONSTANTE_EULER, DENSIDAD_AZUFRE.



Fundamentos de programación en Java

Palabras Reservadas / Claves

La sintaxis de un lenguaje define cómo se usarán las palabras claves, los operadores y las variables para construir y evaluar expresiones. En Java, el conjunto de palabras reservadas es el siguiente:

abstract	char	double	for	int	private	strictfp	throws
boolean	class	else	goto	interface	protected	super	transient
break	const	extends	if	long	public	switch	try
byte	continue	final	implements	native	return	synchronized	void
case	default	finally	import	new	short	this	volatile
catch	do	float	instanceof	package	static	throw	while

true, false, null: no son palabras claves, pero son identificadores de valores reservados, así que tampoco pueden utilizarse como identificadores.

Tipos Primitivos de Datos y Variables



Tipos Primitivos en Java

Java define 8 tipos primitivos fundamentales que almacenan valores simples directamente en memoria:

Tipo	Tamaño	Rango	Uso
byte	8 bits	-128 a 127	Números enteros pequeños, ahorro de memoria
short	16 bits	-32,768 a 32,767	Enteros de rango medio
int	32 bits	-2,147,483,648 a 2,147,483,647	Enteros estándar, tipo más común
long	64 bits	$\pm 9,223,372,036,854,775,808L$	Enteros muy grandes (sufijo L)
float	32 bits	$\pm 3.4 \times 10^{38} F$	Decimales de precisión simple (sufijo f), 6-7 dígitos decimales
double	64 bits	$\pm 1.8 \times 10^{308}$	Decimales de precisión doble, tipo estándar, 15 dígitos decimales
char	16 bits	0 a 65,535	Caracteres Unicode individuales
boolean	1 bit	true false	Valores lógicos verdadero/falso

Declaración y Uso de Variables



```
// Declaración simple  
int edad;  
double salario;  
boolean esEstudiante;
```

```
// Declaración con inicialización  
int contador = 0;  
double precio = 99.99;  
char inicial = 'J';  
boolean activo = true;
```

```
// Múltiples variables del mismo tipo  
int x = 5, y = 10, z = 15;
```

```
// Constantes (inmutables)  
final double PI = 3.14159;  
final int MAX_INTENTOS = 3;
```

```
// Variables de diferentes tipos  
String nombre = "María";  
int edad = 25;  
double altura = 1.65;  
boolean trabaja = false;
```

```
// Conversión de tipos (casting)  
double decimal = 9.8;  
int entero = (int) decimal; // 9
```

Operadores y Expresiones

Operadores Aritméticos

- + Suma
- - Resta
- * Multiplicacion
- / Division
- % Modulo (residuo)
- ++ Incremento
- -- Decremento

```
int a = 10, b = 3;
int suma = a + b; // 13
int resto = a % b; // 1
a++; // a = 11
```

Operadores Relacionales

- == Igual a
- != Diferente de
- > Mayor que
- < Menor que
- >= Mayor o igual
- <= Menor o igual

```
int x = 5, y = 8;
boolean resultado;
resultado = x > y; // false
resultado = x != y; // true
resultado = x <= 5; // true
```

Operadores Lógicos

- && AND lógico
- || OR lógico
- ! NOT lógico

```
boolean a = true, b = false;
boolean r1 = a && b; // false
boolean r2 = a || b; // true
boolean r3 = !a; // false
```

```
// Evaluación de cortocircuito
if (x > 0 && y/x > 2) {
    // y/x solo se evalúa si x > 0
}
```

Operadores de Asignación (=)

```
int num = 10;

num += 5;    // num = num + 5 (15)

num -= 3;    // num = num - 3 (12)

num *= 2;    // num = num * 2 (24)

num /= 4;    // num = num / 4 (6)

num %= 4;    // num = num % 4 (2)
```

Operador Ternario (?)

```
// Sintaxis:
// condición ? valorSiTrue : valorSiFalse
int edad = 20;
String mensaje = (edad >= 18)
                 ? "Mayor de edad"
                 : "Menor de edad";

int max = (a > b) ? a : b;
```

La **precedencia de operadores** determina el orden de evaluación: primero paréntesis, luego multiplicación/división, después suma/resta, seguido por relacionales, y finalmente lógicos.

Usa paréntesis para clarificar expresiones complejas.

Introducción a Estructuras de Control y Colecciones

- Esta sección sirve como una introducción general a los conceptos fundamentales de programación.
- Los temas a cubrir serán: estructuras condicionales para la toma de decisiones, bucles para la repetición de tareas, arrays para almacenar colecciones de elementos de tamaño fijo, y colecciones más dinámicas como listas y mapas.
- Aquí presentamos ejemplos muy básicos para familiarizarnos con ellos.

Estructuras de Flujo de Control

Condicionales

```
// if-else básico
if (condicion) {
    // Código si es verdadero
}
else {
    // Código si es falso
}
```

Bucles

```
// for básico
for (int i = 0; i < 5; i++) {
    // Código a repetir 5 veces
}
```

Estructuras Condicionales en Java



Sentencia `if-else`

Ejecuta un bloque de código si una condición booleana es verdadera. Puede combinarse con `else if` para múltiples condiciones y `else` para un bloque por defecto.

```
int temperatura = 25;
if (temperatura > 30) {
    System.out.println("Mucho calor");
} else
    if (temperatura > 20) {
        System.out.println("Agradable");
    } else {
        System.out.println("Frío");
    }
```

- **Uso:** Para lógica compleja, condiciones múltiples o rangos de valores.
- **Mejor Práctica:** Asegurarse de cubrir todas las posibles condiciones, considerar el orden de evaluación



Sentencia `switch`

Permite seleccionar uno de varios bloques de código a ejecutar basándose en el valor de una variable (`int`, `byte`, `short`, `char`, `String`, o `enum`).

```
String diaSemana = "Martes";
switch (diaSemana) {
    case "Lunes":
        System.out.println("Inicio semana");
        break;
    case "Martes": case "Miércoles":
        System.out.println("Mitad semana");
        break;
    default:
        System.out.println("Fin de semana");
}
```

- **Uso:** Para selecciones basadas en un valor discreto de una variable.
- **Mejor Práctica:** Siempre incluir `break` después de cada `case` (a menos que se desee "fall-through") y un `default`.



Operador Ternario

Una forma concisa de escribir una sentencia `if-else` simple que devuelve un valor. Ideal para asignaciones condicionales en una sola línea.

```
int edad = 17;

String estado = (edad >= 18) ?
    "Mayor de edad" : "Menor de edad";

System.out.println(estado);

// Salida: Menor de edad

int num1 = 10, num2 = 20;

// max será 20

int max = (num1 > num2) ? num1 : num2;
```

- **Uso:** Para expresiones condicionales simples y asignaciones directas.
- **Mejor Práctica:** Evitar la anidación excesiva para no comprometer la legibilidad del código.

Mg. Karens Medrano

Introducción a las Estructuras de Repetición

- Las *estructuras de repetición*, comúnmente conocidas como *bucles*, son herramientas fundamentales en la programación que permiten ejecutar un conjunto de instrucciones de manera repetida.
- Son esenciales para automatizar tareas y procesar colecciones de datos de forma eficiente.
- En Java, se cuenta con diferentes tipos de bucles, incluyendo el bucle `for`, los bucles `while` y `do-while`, y cómo controlar su flujo.

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Hola mundo");  
}
```


Bucle `for`: Iteración Controlada y Eficiente

Bucle `for` Tradicional

Utilizado cuando el número de iteraciones es conocido de antemano. Proporciona un control explícito sobre la inicialización, la condición de continuación y la actualización de la variable de control.

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Paso " + (i + 1));  
}  
// Iterando un array por índice  
String[] dias = {"Lun", "Mar", "Mié"};  
for (int i = 0; i < dias.length; i++) {  
    System.out.println(dias[i]);  
}
```

- **Sintaxis:**
`for (inicialización; condición; actualización)`
- **Uso:** Iterar sobre índices de arrays, ejecutar código un número fijo de veces, operaciones que requieren el índice.
- **Mejor Práctica:** Asegurarse de que la condición de parada se alcance para evitar bucles infinitos. Declarar la variable de control (`i`) dentro del bucle para limitar su alcance.

Bucle `for-each` (For Mejorado)

Proporciona una forma más concisa de iterar sobre todos los elementos de colecciones (arrays, listas, etc.) sin la necesidad de gestionar índices. Simplifica el código, especialmente cuando el índice no es relevante.

```
List<String> frutas =  
    Arrays.asList("Manzana", "Pera", "Uva");  
for (String fruta : frutas) {  
    System.out.println(fruta);  
}  
// Iterando un array directamente  
int[] numeros = {10, 20, 30};  
for (int num : numeros)  
    System.out.println(num);
```

- **Sintaxis:**
`for (TipoDato elemento : coleccion)`
- **Uso:** Recorrer todos los elementos de un array o colección.
Leer valores de una secuencia.
- **Mejor Práctica:** Ideal para lectura de elementos. No permite modificar la colección directamente durante la iteración, ni iterar en orden inverso o saltar elementos específicos.

Bucles `while` y `do-while`: Flexibilidad Condicional

Bucle `while`

Ejecuta la repetición de un bloque de código **mientras** su condición booleana sea verdadera.

La condición se evalúa **antes** de cada iteración, lo que significa que si la condición es falsa desde el principio, el bloque de código nunca se ejecuta.

```
int contador = 0;
while (contador < 3) {
    System.out.println("Contador: " + contador);
    contador++;
}
```

- **Uso Principal:** Cuando el número de iteraciones es desconocido y depende de una condición que puede ser falsa inicialmente (cero o más ejecuciones).
- **Ejemplo:** Leer datos de un archivo hasta el final, esperar la entrada de un usuario válida.

Bucle `do-while`

A diferencia de `while`, el bucle `do-while` garantiza que el bloque de código se ejecute **al menos una vez**.

La condición se evalúa **después** de la primera iteración, y luego, si es verdadera, el bucle continúa.

```
int opcion;
do {
    System.out.println("1. Jugar\n2. Salir");
    opcion = new Scanner(System.in).nextInt();
} while (opcion != 1 && opcion != 2);
```

- **Uso Principal:** Cuando se necesita asegurar que el bloque se ejecute al menos una vez (una o más ejecuciones).
- **Ejemplo:** Presentar un menú al usuario, solicitar una contraseña hasta que sea correcta.

Control de Flujo en Bucles: Optimizando la Ejecución



break

Termina el bucle (for, while, do-while) inmediatamente. La ejecución continúa en la instrucción siguiente al bucle.

```
for (int i = 0; i < 5; i++) {  
    if (i == 2) break;  
    System.out.println(i); // Imprime solamente 0, 1  
}
```

- **Uso:** Salir de un bucle al encontrar un elemento o una condición de error.
- **Práctica:** Útil para evitar procesamiento innecesario una vez que el objeto bucle se ha cumplido.

return

No solo termina el bucle, sino que también sale completamente del método que lo contiene, devolviendo un valor si el método no es void.

```
public boolean buscar(int[] arr, int val) {  
    for (int n : arr) {  
        if (n == val) return true;  
    }  
    return false;  
}
```

- **Uso:** Cuando el hallazgo dentro del bucle significa que el método ha completado su propósito.
- **Práctica:** Usar cuando el resultado del bucle es crítico para la salida del método.

continue

Omite la iteración actual del bucle y procede a la siguiente iteración. El código restante dentro del cuerpo del bucle para esa iteración no se ejecuta.

```
for (int i = 0; i < 5; i++) {  
    if (i % 2 == 0) continue;  
    System.out.println(i); // Imprime 1, 3  
}
```

- **Uso:** Saltar elementos que no cumplen un criterio o condiciones no válidas.
- **Práctica:** Mejora la legibilidad al evitar anidamientos profundos para casos excepcionales.

Vectores y Colecciones



- Son contenedores de objetos en Java que almacenan colecciones de elementos del mismo tipo.
- Los arrays (vectores) son estructuras de datos que permiten almacenar múltiples valores del mismo tipo en una secuencia de tamaño fijo.
- Son estructuras fundamentales para organizar datos de manera eficiente, especialmente cuando se conoce la cantidad de elementos con antelación.
- Las colecciones, por otro lado, ofrecen estructuras más flexibles y potentes para organizar datos, como listas que pueden cambiar de tamaño dinámicamente o mapas que asocian valores a claves.

Arrays (Vectores)



UNIVERSIDAD
DON BOSCO

Concepto Básico



```
int[] numeros; // Declaración  
numeros = new int[5]; // Inicialización (tamaño fijo de 5 enteros)
```

- **Homogéneo:** Almacena solo un tipo de datos.
- **Tamaño Fijo:** Su longitud no puede cambiar después de la creación.

Declaración e Inicialización



```
// Declarar y crear un array de 3 elementos  
String[] nombres = new String[3];  
// Declarar y inicializar con valores  
double[] temperaturas = {25.5, 26.1, 24.9};  
// Acceder a la longitud del array  
int longitud = temperaturas.length; // longitud es 3
```

Acceso y Modificación de Elementos



```
// Acceder a un elemento  
String primerNombre = nombres[0]; // "Ana" si lo inicializamos previamente  
nombres[0] = "Sofía"; // Modificar un elemento  
// Recorrer un array (bucle for)  
for (int i = 0; i < nombres.length; i++) {  
    System.out.println(nombres[i]);  
}
```

Arrays (Vectores)



Utiliza arrays cuando el tamaño de la colección sea fijo y conocido, y cuando necesites trabajar con tipos de datos primitivos.

Arrays Multidimensionales

Un array multidimensional es un "array de arrays". El más común es el bidimensional (matriz), que se representa como filas y columnas.

Son útiles para representar tablas o coordenadas.



```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6}};
// Acceder al elemento en la fila 0, columna 1
int valor = matriz[0][1]; // valor es 2
```

Métodos Útiles (Clase Arrays)

La clase `java.util.Arrays` proporciona métodos estáticos muy útiles para manipular arrays.



```
import java.util.Arrays;
int[] numeros = {5, 2, 8, 1};
Arrays.sort(numeros); // numeros ahora es {1, 2, 5, 8}
String strArray = Arrays.toString(numeros); // strArray es "[1, 2, 5, 8]"
int[] copia = Arrays.copyOf(numeros, 2); // copia es {1, 2}
```

Arrays vs. ArrayLists

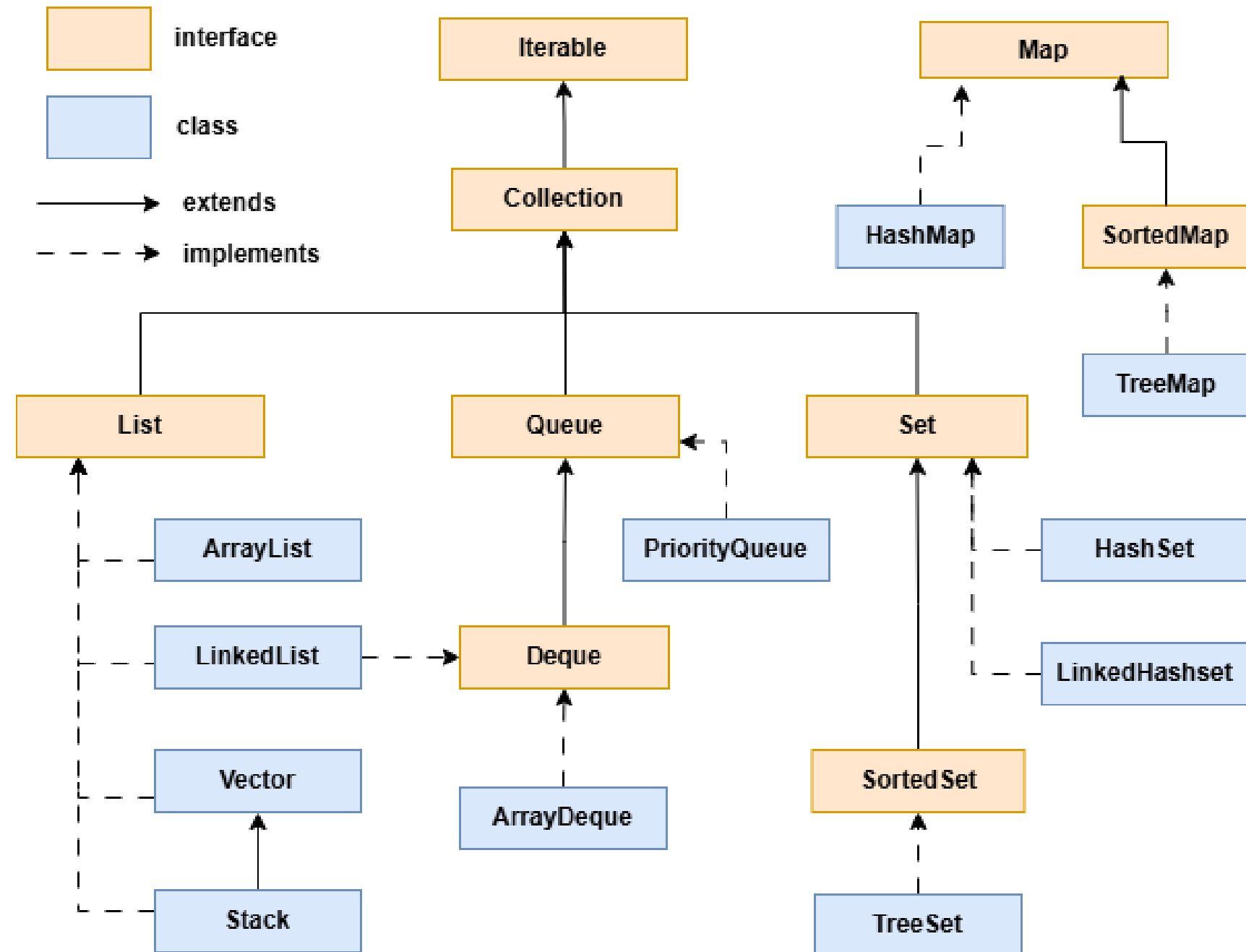
Aunque similares, tienen diferencias clave:



- **Arrays:** Tamaño fijo, pueden almacenar primitivos y objetos, rendimiento generalmente más rápido para acceso directo.
- **ArrayLists:** Tamaño dinámico (se ajusta automáticamente), solo almacenan objetos (`Integer` en lugar de `int`).

Colecciones en Java(1)

- El *Java Collections Framework* es una arquitectura unificada para representar y manipular *colecciones de objetos*.
- Java brinda en el paquete **java.útil** a un juego de *clases* e *interfaces* para guardar colecciones de objetos. En él, todas las entidades conceptuales están representadas por **interfaces** y las **clases** se usan para proveer implementaciones de esas interfaces.
- Una **interfaz** dice qué podemos hacer con un objeto. Un objeto que cumple determinada interfaz es “algo con lo que puedo hacer X”.
- La interfaz no es la descripción entera del objeto, solamente un mínimo de funcionalidad con la que debe cumplir



Colecciones en Java(2)



Interfaces mas comunes

Collection

- Una Collection es todo aquello que se puede recorrer (o iterar) y de lo que se puede saber el tamaño. Muchas otras clases extenderán la interfaz Collection imponiendo más restricciones y dando más funcionalidades.

```
Collection <clase_elementos> nombre_objeto = new ClaseDeCollection<clase_elementos> ( );
```

List

- Es una implementación de la interface Collection. Define una sucesión de elementos donde cada uno de ellos lleva un índice asociado.
- Por ej. Puede definir una lista con nombres de personas, cada uno asociado a un índice y puede cambiar su contenido:

```
usuario -> ( Jorge López [0], Andrea Maldonado [1], Luis Pérez [2], Elías Sánchez [3], Mirna Acosta [4] )
```

- Sintaxis:

```
List <clase_elementos> nombre_objeto = new ClaseDeList<clase_elementos> ( );
```

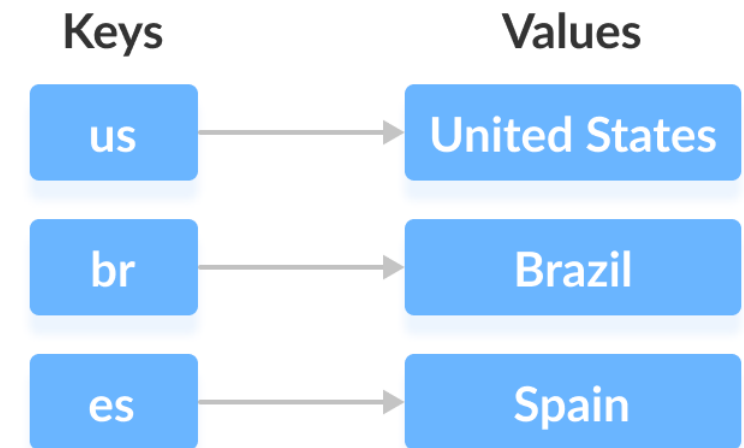
Colecciones en Java(3)

Interfaces mas comunes

Map

- Gestiona lo que en otros lenguajes se conoce como “diccionario” y que suele asociarse a la idea de “tabla hash”.
- Un Map es un conjunto de valores, en donde c/valor tiene un objeto extra asociado. A los primeros se los llama “claves” o “keys”, ya que son los que permiten acceder a los segundos.
- Un Map no es una Collection ya que esa interfaz es limitada. Se puede afirmar que Collection es unidimensional, mientras que un Map es bidimensional.
- Su sintaxis es la siguiente:

```
Map <clase_claves, clase_valores> nombre_objeto =  
    new Map< clase_claves, clase_valores > ( );
```



Colecciones en Java(4)



Clase ArrayList

Un `ArrayList` es una implementación de la interfaz `List` que utiliza un array dinámico para almacenar sus elementos.

- **Características:** Permite elementos duplicados, mantiene el orden de inserción, acceso rápido por índice.
- **Rendimiento:**
 - Acceso (`get`): $O(1)$ - muy eficiente.
 - Adición al final (`add`): $O(1)$ en promedio.
 - Inserción/ Eliminación en medio (`add(index, element)`, `remove(index)`):
 $O(n)$ - lento, ya que requiere desplazar elementos.

```
ArrayList<String> listaNombres = new ArrayList<>();  
listaNombres.add("Ana");  
listaNombres.add("Juan");  
System.out.println(listaNombres.get(0)); // obtiene copia de valor: Ana
```

Colecciones en Java(5)



Clase LinkedList

Implementa las interfaces `List` y `Deque`. Almacena elementos en nodos, donde cada nodo contiene el elemento y referencias al nodo anterior y siguiente.

- **Características:** Permite duplicados, mantiene el orden de inserción, flexible para inserciones/eliminaciones.
- **Uso Común:** Cuando se realizan muchas inserciones o eliminaciones en el medio de la lista, o cuando se usa como cola (Queue) o pila (Stack).
- **Rendimiento:**
 - Acceso (`get`): $O(n)$ - lento, ya que debe recorrer la lista.
 - Inserción/Eliminación (`add`, `remove`): $O(1)$ en los extremos, $O(n)$ en el medio (después de encontrar la posición).

```
LinkedList<Integer> numeros = new LinkedList<>();  
numeros.add(10);  
numeros.addFirst(5); // Agrega al inicio  
System.out.println(numeros.removeLast()); // Remueve el último (10)
```

Colecciones en Java(6)

Clase HashSet

Implementa la interfaz `Set` y se basa en una tabla hash para almacenar sus elementos. Garantiza que no haya elementos duplicados.

- **Características:** No permite duplicados, no mantiene el orden de inserción, permite un único valor `null`.
- **Uso Común:** Para almacenar colecciones de elementos únicos donde el orden no es importante y se necesita una búsqueda, adición y eliminación muy rápidas.
- **Rendimiento:**
 - Operaciones básicas (`add`, `remove`, `contains`): $O(1)$ en promedio, suponiendo una buena función hash.
 - Peor caso: $O(n)$ si hay muchas colisiones en la tabla hash.

```
HashSet<String> frutas = new HashSet<>();  
frutas.add("Manzana");  
frutas.add("Pera");  
frutas.add("Manzana"); // No se añade, ya existe  
System.out.println(frutas.contains("Pera")); // true
```

Colecciones en Java(7)

Clase HashMap

Un `HashMap` implementa la interfaz `Map` y almacena pares clave-valor. Es ideal para buscar un valor rápidamente a partir de una clave.

- **Características:** Las claves deben ser únicas (un valor `null` como clave es posible), los valores pueden ser duplicados. No mantiene el orden de inserción.
- **Uso Común:** Para asociaciones de datos, como un diccionario o tabla de búsqueda, donde se necesita recuperar un valor rápidamente por su clave.
- **Rendimiento:**
 - Operaciones básicas (`put`, `get`, `remove`, `containsKey`): $O(1)$ en promedio.
 - Peor caso: $O(n)$ si hay muchas colisiones hash.

```
HashMap<String, Integer> edades = new HashMap<>();  
edades.put("Carlos", 30);  
edades.put("Maria", 25);  
System.out.println(edades.get("Carlos")); // 30
```

Colecciones en Java(8)



- Además de estas, Java ofrece otras colecciones como **LinkedHashSet** (mantiene orden de inserción en un Set), **LinkedHashMap** (mantiene orden en un Map), **TreeMap** (ordena claves en un Map), y **TreeSet** (ordena elementos en un Set).
- La elección correcta depende de los requisitos específicos de orden, unicidad, acceso y rendimiento de tu aplicación.

Ejercicios Prácticos de Java



- ¡Es hora de poner en práctica lo aprendido!
- A continuación, se presentan una serie de ejercicios de programación en Java diseñados para reforzar tu comprensión de los conceptos desde lo más básico hasta las estructuras de datos avanzadas. Intenta resolverlos para consolidar tus conocimientos.



1. Calculadora Básica

Escribe un programa que solicite al usuario dos números y una operación aritmética (+, -, *, /).

El programa debe realizar la operación y mostrar el resultado. Asegúrate de manejar la división por cero.



2. Verificador de Números Primos

Desarrolla un programa que determine si un número entero positivo ingresado por el usuario es un número primo.

Un número primo es aquel que solo es divisible por 1 y por sí mismo.



3. Invertir una Cadena

Crea un método que tome una cadena de texto (String) como entrada y devuelva una nueva cadena con los caracteres en orden inverso, sin usar métodos preexistentes de inversión de cadenas.



4. Gestión Simple de Tareas

Implementa un programa que permita al usuario añadir, ver y eliminar tareas de una lista. Utiliza un ArrayList para almacenar las tareas.

El programa debe presentar un menú de opciones.

Sugerencia: Un bucle while para el menú, y métodos como add(), get() y remove() de ArrayList.



5. Inventario Básico de Productos

Diseña un sistema de inventario donde cada producto tiene un ID único (String) y un nombre (String).

El usuario debe poder añadir nuevos productos, buscar un producto por su ID y mostrar todos los productos. Utiliza un HashMap.

Sugerencia: Usa HashMap<String, String> para clave (ID) y valor (nombre del producto). Implementa put() y get().

Bibliografía y Recursos Adicionales

El aprendizaje de Java es un viaje continuo. Aquí te presentamos una selección de recursos clave, organizados por tipo y enfoque, para que puedas profundizar tus conocimientos, mantenerte al día con las últimas tendencias y construir una base sólida para tu carrera como desarrollador.

Libros Esenciales



Fundamentos sólidos y guías avanzadas para dominar el lenguaje.

- **"Thinking in Java" (Bruce Eckel):** Un clásico para entender a fondo la programación orientada a objetos en Java. (Nivel: Intermedio-Avanzado)
- **"Effective Java" (Joshua Bloch):** Consejos y mejores prácticas para escribir código Java eficiente y de alta calidad. (Nivel: Intermedio-Avanzado)
- **"Head First Java" (Kathy Sierra & Bert Bates):** Una introducción visual y amigable para principiantes. (Nivel: Principiante)
- **Clean Code** de Robert C. Martin: Principios para escribir código limpio y mantenible.

Documentación Oficial y Sitios Web



Fuentes autorizadas para consulta y referencia constante.

- **Documentación Oficial de Oracle Java:** La fuente definitiva para la API de Java, especificaciones y tutoriales oficiales. (docs.oracle.com/en/java/)
- **OpenJDK Project:** Para entender las implementaciones de Java y contribuir a su desarrollo. (openjdk.java.net)
- **Baeldung:** Gran cantidad de tutoriales y guías prácticas sobre Java, Spring y ecosistema relacionado. (www.baeldung.com)

Canales de YouTube



Explicaciones visuales, resolución de problemas y consejos de expertos.

- **Telusko:** Tutoriales de Java, Spring Boot y desarrollo en general, con buen equilibrio entre teoría y práctica.
- **Programación ATS:** Contenido en español con cursos completos de Java desde cero, POO, estructuras de datos, etc.
- **Code with Mosh:** Explicaciones claras y concisas sobre diversos temas de programación, incluyendo Java.

Recuerda que la clave del éxito en la programación es la **práctica constante** y la **curiosidad por aprender**.
¡No dudes en explorar estos recursos y encontrar los que mejor se adapten a tu estilo de aprendizaje!

Agradecimientos y Cierre

¡Enhorabuena por completar este módulo introductorio de Java! Has dado un paso fundamental para dominar uno de los lenguajes de programación más poderosos y versátiles del mundo.



Continúa Aprendiendo

El mundo de Java evoluciona constantemente. Mantén tu curiosidad, explora nuevas bibliotecas, frameworks y patrones de diseño. Cada nuevo conocimiento te abrirá más puertas.



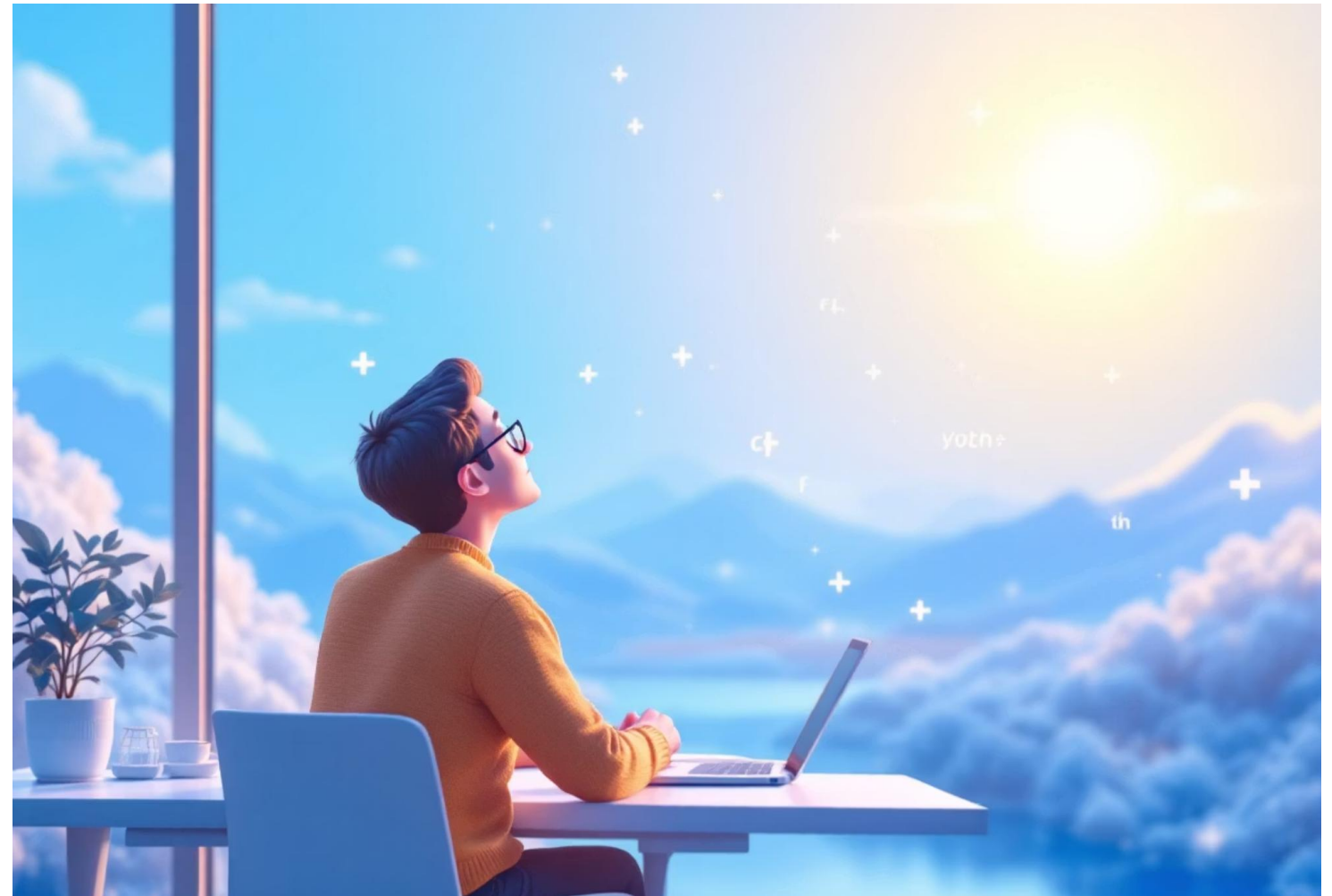
Conceptos Clave

Recuerda los pilares: programación orientada a objetos, gestión de memoria, estructuras de datos y algoritmos. Estos son los cimientos sobre los que construirás soluciones complejas.



La Práctica es Clave

La verdadera maestría se alcanza con la práctica constante. Escribe código a diario, resuelve problemas, contribuye a proyectos y no temas experimentar. ¡Así es como se aprende de verdad!



Tu viaje como desarrollador Java acaba de comenzar. Te animamos a seguir construyendo, innovando y transformando ideas en realidad. ¡El futuro está en tus manos!